Triangle Solver – Test Plan

Project: Polygon Checker
Group: CSCN71020_Group4
Prepared by: Dora Ahmed
Date: 2025-11-05

1. Test Objectives

The purpose of this Test Plan is to confirm the functionality and accuracy of the Triangle Solver module within the Polygon Checker Project. The objectives are to:

- Validate the program correctly identifies whether three sides form a valid triangle.
- Classify valid triangles as Equilateral, Isosceles, or Scalene.
- Further classify triangles based on internal angles as Right-Angled, Acute, or Obtuse
- Handle invalid or non-triangle inputs smoothly without crashing the system

2. Test Scope

This test plan focuses on both Triangle Classification by sides and Classification by Angles in the Polygon Checker application.

Functions in Scope

- analyzeTriangle(a, b, c)
- classifyByAngles(A,B,C)

Inputs

- three numerical side lengths(a, b, c)

Outputs

Text classification such as:

- “Equilateral Triangle – Acute”
- “Isosceles Triangle – Acute”
- “Scalene Triangle – Right-Angled”
- “Not a Triangle”

Out of Scope

- Rectangle and other non-triangle shapes.

- User Interface and Input/Output[forms] formats

3. Test Environment

Component	Description
IDE	Visual Studio 2022
Language	C
Testing Framework	Microsoft Native C++ Unit Test Framework
Source Control	GitHub (CSCN71020_Group4 repository)
OS	Windows 10/11

4. Test Cases

Test ID	Test Description	Input (a,b,c)	Expected Output	Pass Criteria	Fail Criteria
TC1	Invalid – Negative Side	-2, 5, 6	Not a Triangle	Negative input handled correctly	Program produces triangle classification
TC2	Invalid – Zero Side	0, 4, 5	Not a Triangle	Program correctly identifies valid side	Accept invalid side or crashes
TC3	Invalid – Violates Inequality	1, 2, 3	Not a Triangle	Output exactly matches expected string	Output differs or function crashes
TC4	Invalid – All Zero	0, 0, 0	Not a Triangle	Output string matches expected message	Any other input
TC5	Invalid – Boundary Sum	2, 3, 5	Not a Triangle	Properly rejects edge case	Incorrect classification
TC6	Equilateral Triangle	3, 3, 5	Equilateral Triangle - Acute	Output contains both “Equilateral” and “Acute”	Missing or wrong labelling

TC7	Isosceles Triangle	3, 3, 4	Isosceles Triangle - Acute	Output matches type and angle	Any mismatch input
TC8	Scalene Triangle	4, 5, 6	Scalene Triangle - Acute	Function returns both labelling	Returns only one or incorrect
TC9	Right – Angled Triangle	3, 4, 5	Scalene Triangle – Right Angled	Confirms right-angle detection	Fails to detect 90°
TC10	Acute Triangle	6, 7, 8	Scalene Triangle - Acute	Output contains “Acute”	Output missing
TC11	Obtuse Triangle	2, 3, 4	Scalene Triangle - Obtuse	Correctly detects >90° angle	Wrong output

5. Expected Results Summary

Triangle Type	Condition	Expected Output
Equilateral	a == b == c	“Equilateral Triangle – Acute”
Isosceles	Two equal sides	“Isosceles Triangle – Acute”
Scalene (Right)	$a^2 + b^2 = c^2$	“Scalene Triangle – Right – Angled”
Scalene (Acute)	All angles <90°	“Scalene Triangle – Acute”
Scalene (Obtuse)	One angle >90°	“Scalene Triangle – Obtuse”
Invalid	Fails validation	“Not a Triangle”

6. Pass/Fail Criteria

Criterion	Description
Pass	All test cases compile, execute, and output exactly the expected classification strings

	without logical errors.
Fail	Output strings differ from expected results, missing clarification, or the program crashes due to unhandled input.

Rectangle Feature – Test Plan

Project: Polygon Checker
Group: CSCN71020_Group4
Prepared by: Dora Ahmed and Oluwaloni Ayeni
Date: 2025-11-11

7. Rectangle Feature Development

Purpose:

The purpose of this section is to describe the development and accuracy of the rectangle feature added to the polygon Checker project.
This feature determines whether four given coordinate points form a valid rectangle and, if valid, calculates its area and perimeter.

8. Objectives

- Validate that the input points form a valid rectangle using coordinate geometry.
- Use the distance formula to calculate the lengths of all four sides.
- Use the dot-product formula to confirm that all angles are 90°
- Calculate and verify the area and perimeter of valid rectangles.
- Handle invalid inputs gracefully without crashing.

Functions in Scope

- isRectangle(x1, y1, x2, y2, x3, y3, x4, y4)
- calculateArea(x1, y1, x2, y2, x3, y3, x4, y4)
- calculatePerimeter(x1, y1, x2, y2, x3, y3, x4, y4)

9. Test Environment

Component	Description
IDE	Visual Studio 2022

Language	C
Testing Framework	Microsoft Native C++ Unit Test Framework
Source Control	GitHub (CSCN71020_Group4 repository)
OS	Windows 10/11

10. Test Cases

Test ID	Test Description	Input (x1, y1, x2, y2, x3, y3, x4, y4)	Expected Output	Pass Criteria	Fail Criteria
RC1	Valid rectangle (points form right angles)	(2,2), (8,2), (8,6), (2,6)	“Valid rectangle – Area = 24 Perimeter = 20”	All angles = 90°, opposite sides equal	Incorrect area/ perimeter values
RC2	Not a rectangle (points cross over)	(1,1), (7,1), (6,5), (2,5)	“Invalid rectangle”	Detects shape mismatch	Incorrect valid returns
RC3	Square (special case rectangle)	(0,0), (4,0), (4,4), (0,4)	“Valid rectangle – Area = 16 Perimeter = 16”	Equal sides, all right angles	Fails to detect square
RC4	Collinear points	(0,0), (2,0), (4,0), (6,0)	“Not a rectangle”	Rejects linear input	Accepts line as valid shape
RC5	Duplicate points	(0,0), (0,0), (4,3), (0,3)	“Invalid input – duplicate points”	Detects duplicate coordinates	Accepts or crashes

11. Expected Results Summary

Rectangle Type	Condition	Expected Output
Regular Rectangle	Valid coordinates forming 90° angles and equal opposite sides	“Valid rectangle – Displays area and perimeter”
Square (special case)	All sides equal and 90° angles	“Valid rectangle – Displays equal sides, correct area & perimeter

Invalid Rectangle	Uneven or crossing coordinates	“Invalid rectangle – Shape not detected”
Collinear points	All points on a straight line	“Not a rectangle”
Duplicate points	Overlaps coordinate values	“Invalid input – duplicate points”

12. Pass and Fail Criteria

Criterion	Description
Pass	All rectangle test cases compile, execute, and output the correct validation messages (e.g., “Valid rectangle”, “Invalid rectangle”, “Not a rectangle”) and calculated area/perimeter values without errors.
Fail	Output messages differ from expected results, area/ perimeter calculations are incorrect, or the program crashes or wrong labelling an invalid shape